STUDENT COURSE MANAGEMENT SYSTEM

A COMPREHENSIVE C++ APPLICATION USING OOP & STL

Prepared by: Dalia Mahmoud Elshamy

ID: 12239

Course: Data Structures & Algorithms

PERSONAL VISION

- Objective: To build a scalable and efficient system for managing student records and academic performance.

- Key Goals:

- Implementing Clean Code principles.
- Applying Object-Oriented Programming (OOP) concepts.
- Utilizing C++ Standard Template Library (STL) for data management.

KEY FEATURES

- Student Management: Seamlessly Add, Update, and Remove student records.
- Academic Tracking: Manage courses, credit hours, and grade calculation.
- Smart Sorting: Rank students by GPA or alphabetical order.
- Instant Search: Retrieve student profiles using unique IDs.
- Data Persistence: Automatic saving and loading of data from external files.

TECHNICAL ARCHITECTURE

- Programming Language: C++
- Development Paradigm: Object-Oriented Programming (OOP)
- Core Logic:

  -Inheritance: Base class Person extended by the Student class.

  -Encapsulation: Data protection using private members and public accessors.

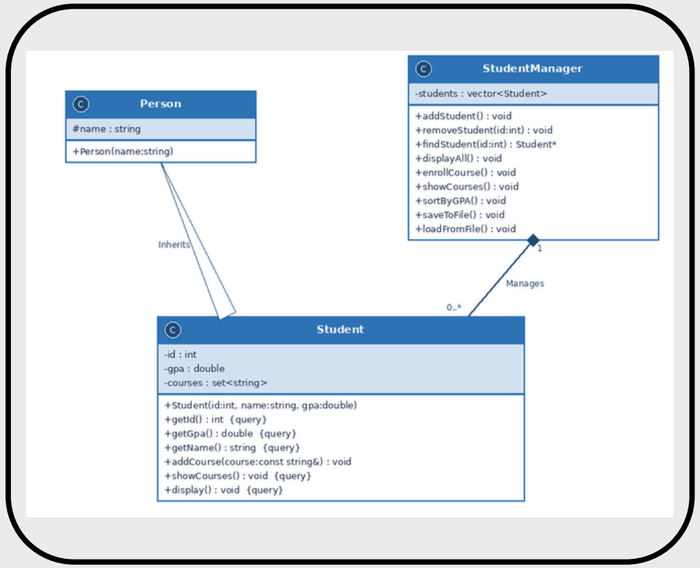
  -Polymorphism: Utilizing virtual functions for specialized behaviors.

DATA STRUCTURES & STL UTILIZATION

- CONTAINERS:
  - STD::VECTOR: FOR DYNAMIC STORAGE AND EFFICIENT INDEXING OF STUDENT LISTS.
  - STD::SET: TO ENSURE UNIQUENESS OF IDS AND PREVENT DATA DUPLICATION.

- ALGORITHMS:
  - STD::SORT: FOR HIGH-PERFORMANCE RANKING.
  - STD::FIND_IF: FOR OPTIMIZED SEARCHING LOGIC.

SYSTEM DESIGN (UML DIAGRAM)



Design Overview:

- Student Class: Encapsulates personal and academic attributes.
- StudentManager Class: The "Control Center" that handles the system's logic, file I/O, and user operations.

USER INTERFACE & EXPERIENCE

- Interface Design:
  - User-friendly console-based menu.
  - Strict Input Validation to prevent system crashes and ensure data integrity.
  - Formatted Table Output for clear and readable data visualization.

CODE EXECUTION (DEMO SCREENSHOTS)

```
void StudentManager::addStudent() {
    int id;
    string name;
    double gpa;

    cout << "Enter ID: ";

    while (!(cin >> id)) {
        cout << "Invalid input. Please enter a numeric ID: ";
        cin.clear();
        cin.ignore(1000, '\n');
    }

    if (findStudent(id) != nullptr) {
        cout << "Student ID already exists" << endl;
        return;
    }
```

```
void StudentManager::saveToFile() {
    ofstream outFile("students_db.txt");
    if (!outFile) {
        cout << "Error opening file for saving" << endl;
        return;
    }
```

```
void StudentManager::sortByGPA() {
    sort(students.begin(), students.end(), [](Student& a, Student& b) {
        return a.getGpa() > b.getGpa();
        });
}
```

- Demonstration:
  - Displaying the main dashboard.
  - Showing successful data entry and calculation logic.

FILE HANDLING & DATA INTEGRITY

- Mechanism: Integrated File I/O system to write data to .txt or .csv files.
- Reliability: Data is automatically synchronized with the storage file upon every session, ensuring no information loss.

CONCLUSION & FUTURE ENHANCEMENTS

- Conclusion: This project successfully demonstrates the practical application of C++ fundamentals in a real-world scenario.

- Future Scope:
  - Transitioning from a CLI to a Graphical User Interface (GUI).
  - Integrating a SQL Database for large-scale data management.

THANK YOU